Group 15 – Grocery Store Staffing Simulation

Zain Usmani, Aariz Farooq, Darryl Nikundana, Ali Mohamad, Jon Aliko, Tarandeep Bhogal

ELEC2320 – Software Fundamentals

Monday, March 2nd, 2026

1. Problem Introduction and Definition

The problem that we are tasked to solve is in relation to a grocery store and its current cashier and check-out staffing process, and the desire to change their process. The grocery store has given us the task of helping to revamp their current system by developing an object-oriented software simulator. This simulator will help to determine if their new process will be beneficial to both customers and their stores. The revamped process that our simulator is tasked to test consists of eight checkout lanes, one marked with a permanent "nine items or less express lane" tag, and one marked with a permanent "nineteen items or less" express tag. Using these lanes, the simulator should test how quickly customers can go through these lanes and pay for their items, with three different configurations to test. These three configurations will be simulated using our designed simulator using the same set of customers and different cashier combinations to measure which configuration is the most efficient. Based on the results from the simulator, this will help the grocery store determine which configuration will be most beneficial to the customers and stores.

2. Requirements

The system's functional requirements define what the simulator does. The system will generate a customer file and a cashier configuration file, then run a simulation that models eight checkout lanes, including two express lanes. The simulator will calculate transaction times based on the item count, payment method, and a small random factor to account for natural variations in processing times. It will also simulate queue formation, handle cashier shift changes, and produce measurable outputs such as customer wait times and cashier idle times. These functions define the required behavior of the simulator.

For the non-functional requirements, the system must run efficiently even when processing large numbers of customers and produce consistent results when the same inputs and random seeds are used. It should validate input files and handle errors cleanly, providing clear and concise reports. The design will follow object-oriented principles to ensure maintainability and scalability, allowing the system to be easily extended in the future, such as adding new payment methods or modifying lane policies. These requirements ensure that the system operates efficiently and reliably while remaining functionally correct.

3. System Design

The grocery checkout simulation system models customer arriving at checkout lanes, waiting in queues, and being served by cashiers. The system consists of six classes, each with a few main responsibilities:

Customer Class:

This class represents a single shopper. It stores all associated information required for the simulation, and provides methods to retrieve customer data, to then be exported in JSON format.

Attributes:
id: unsigned int
arrival_time: unsigned int
item_count: int
arrival_time_string: string
payment_method: paymentMethod

Methods:
Customer(...)
getID()
getItemCount()
getMethodString()
getTimeString()
getTime()
to_json()

Cashier Class:

This class represents a cashier working at a checkout lane. It stores the customers served, queue processing, and service timing.

Attributes:
CashierID: int
startTime: unsigned int
endTime: unsigned int
queue: queue<Customer*>
isBusy: bool
busyUntil: unsigned int
idleTime: unsigned int

Methods:
isAvailable(currentTime)
addCustomer(Customer*)
startService(Customer*)
finishService()
getQueueLength()

CheckoutLane Class:

This class represents a checkout lane within the store. This lane object can be defined as either an express or regular lane. It contains both the “Customer” and “Cashier” classes.

Attributes:
laneID : int
expressLimit : int
cashier : Cashier*
queue : queue<Customer*>

Methods:
canAcceptCustomer(Customer*)
addCustomer (Customer*)
getEstimatedWaitTime()

Store Class:

This class represents the entire grocery store. It manages all checkout lanes by distributing and logging customers in queues.

Attributes:	Methods:
lanes : vector<CheckoutLane*>	assignCustomerToLane(Customer*) updateSimulation(currentTime)

Event Class:

This class represents events taking place such as a customer's arrival as well as service completion.

Attributes:	EventType Enum:
eventTime : unsigned int	ARRIVAL
type : EventType	SERVICE_START
customer : Customer*	SERVICE_END
	CASHIER_START
	CASHIER_END

Simulation Class:

This class controls the entire simulation. It loads input files, processes events, and generates output statistics for the user.

Attributes:	Methods:
store : Store	loadCustomers(file)
eventQueue : priority_queue<Event>	loadCashiers(file)
currentTime : unsigned int	run()
	processEvent(Event)
	generateReport()

Figure 1 - UML Class Diagram

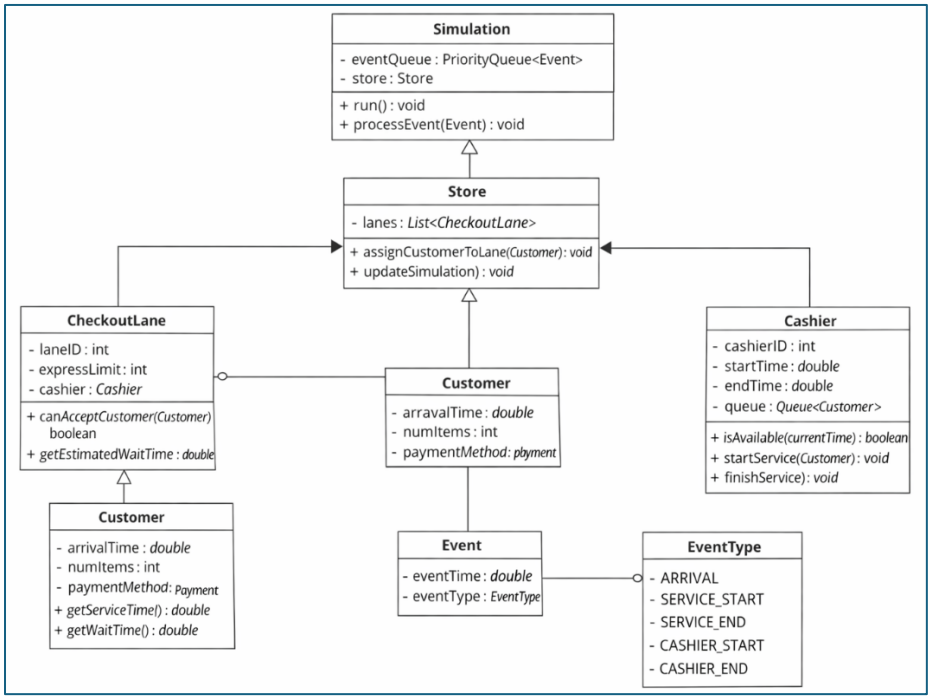
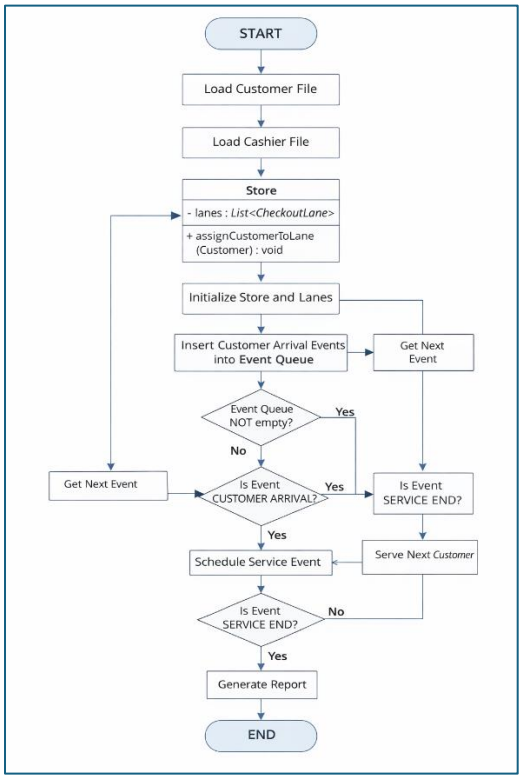


Figure 2 - System Flowchart



4. Algorithms

Generate Random Factor

Purpose

This algorithm introduces realistic human variability into the simulation to ensure that every transaction does not take the exact same amount of time. This value represents the "small random factor" required by the project specifications. It is added to the base transaction time to simulate real world delays such as a dropped coin or a slow receipt printer.

Logic

1. Initialize a random number generator using the system clock as a seed to ensure unique starting values for every execution.
2. Define a normal distribution curve centered at zero.
3. Draw a random floating point value from this normal distribution.
4. Return the generated value to be added to the customer total processing time.

MergeSort

Purpose

When the user enters customers, they aren't automatically in chronological order. This algorithm ensures that the simulation processes customers in the chronological order of their arrival timestamps as outlined in the generated JSON files.

Logic

1. Accept an unsorted vector of generated customer objects.
2. If the array contains one or zero elements, then return the array as it is already sorted.
3. Divide the customer array into two equal halves recursively until only single elements remain.
4. Merge the halves back together by comparing the arrival timestamp of each customer.
5. Place the customer with the earlier timestamp into the new combined list.

Transaction Time Calculation

Purpose

To determine the precise duration a customer occupies a cashier based on their specific purchase and payment data. This will make use of the RandomFactor function mentioned above, as well as the rules defined in the problem statement.

Logic

1. Retrieve the total item_count and payment method from the current customer object.
2. Multiply the item count by the constant 5 seconds to get the base scanning time.

3. Add the hardcoded payment delay time which is 60 seconds for cash or 120 seconds for debit or 150 seconds for cheque.
4. Call the Generate RandomFactor function and add that random value.
5. Use this total time to schedule the customer departure event and update the cashier queue.

Lane Choice Logic

Purpose
To mimic the decision-making process of a customer as they arrive at the checkout area and attempt to minimize their wait.

Logic
1. Identify all checkout lanes that currently have a cashier on duty.
2. Check the customer item count against the 9 item and 19 item express limits.
3. Filter the available lanes to only include regular lanes, and the specific express lanes the customer is eligible to use.
4. Calculate a queue weight for each eligible lane by checking the number of customers currently waiting in its line.
5. Assign the customer to the specific lane with the absolute lowest queue weight.

5. Libraries & Tools

CMake

Link: https://cmake.org

CMake is used to achieve cross platform compatibility. By using it, we can allow users to easily compile all three programs along with any dependencies with just a few, simple commands. It is used to cleanly separate the code of the configuration programs and the simulation engine while linking shared dependencies. It automates the compilation process and ensures the project builds reliably across different operating systems. By standardizing the build pipeline across the team, we prevent environment mismatch errors and ensure that the three independent executables compile seamlessly from a single source directory.

JSON for Modern C++ - Niels Lohmann

GitHub: https://github.com/nlohmann/json

This dependency is used to organize the customer list and staffing configuration data into an organized and easily readable JSON format. Using JSON ensures that the customer lists and staffing configurations are easily readable and structured for the third simulation program.

Dear ImGui

GitHub: https://github.com/ocornut/imgui

Dear ImGui is a fast graphical user interface library for C++. It will be used exclusively for the first two programs to build visual inputs for customer generation and cashier configuration files. Instead of storing a complex graphical object in memory, it redraws the entire GUI from scratch every single frame inside a constant while loop. This makes it lightweight and perfectly suited for generating the GUI of our config programs without the heavy CPU overhead required from traditional graphical frameworks.

GLFW (Supports ImGui)

Link: https://www.glfw.org/documentation.html

Dear ImGui cannot create operating system windows or handle user inputs on its own. GLFW is a lightweight utility library that creates the actual application window and captures mouse and keyboard events so users can interact with the configuration programs. It bridges the gap between the C++ backend logic and the operating system interface.

OpenGL (Supports ImGui)

Link: https://www.opengl.org/

Dear ImGui calculates the math and acts as an architect that draws the visual blueprint for where all the buttons and text should go. It does not have the ability to actually build anything on the screen. Using that analogy, OpenGL is the like “construction worker” that takes this mathematical blueprint and knows exactly how to “build it”, or talk to the computer graphics hardware. It physically paints the pixels and colors onto the screen so the user can see and interact with the GUI.

6. Task Distribution

Zain Usmani: Project Management and Build System

Managed the GitHub repository for team collaboration, and configured CMake to ensure cross-platform compatibility and compiled all three independent exe’s from a single source directory.

Aariz Farooq: User Interfaces (Programs 1 and 2)

Will develop the GUI for the first two programs using Dear ImGui. Planning on integrating GLFW and OpenGL to handle operating system windows, mouse & keyboard inputs, and hardware rendering for the config generator tools.

Darryl Nikundana: Data Serialization

Implemented the nlohmann/json library to structure the customer lists and staffing configuration data. Will ensure the data exported from Programs 1 and 2 will be formatted into a readable JSON format.

Ali Mohamad: Core Entity Architecture (Program 3)

Will develop and design the foundational object-oriented classes representing the core store components such as the Customer, Cashier, and CheckoutLane classes, managing attributes such as service times and queue lengths.

Tarandeep Bhogal: Core Algorithms and Math

Will the system's core mathematical algorithms, including the transaction time calculation and lane choice logic. Implemented the MergSort algorithm and the random factor generator using <random> and <ctime> libraries to simulated real-world delays.

Jon Aliko: Simulation and Event Loop (Program 3)

Currently building the overarching architecture for the simulation program. Implemented the Store class to manage and distribute customers across all checkout lanes. Will develop the Event class to track the simulation timeline, logging customer arrivals and service completions.

7. Conclusion

This design document outlines the complete architectural plan for the Group 15 Grocery Store Staffing Simulation. By separating the project into three distinct programs, we ensure a modular approach to generating customer data, configuring cashier shifts, and executing the core simulation. The object-oriented approach utilizing the Customer, Cashier, and CheckoutLane classes guarantees that the codebase remains scalable and maintainable. Implementing the external JSON library ensures reliable data transfer between the configuration tools and the simulation engine.

Furthermore, the integration of Dear ImGui and GLFW will provide an accessible user interface for the configuration tools. Ultimately, this simulator will process the complex mathematical queues and random variables required to accurately measure customer wait times and cashier idle times. The final software will provide the grocery store management with the data-driven metrics they need to determine if their new configuration will be both beneficial to customers and efficient in staffing.